

Scope, Classes, and Lists

What is scope?

- Variables behave differently when declared outside a function vs inside a function (or class or module)
- If a variable is created inside of a function, then it does not exist outside of that function - we call this a local variable
- If a variable is created outside of a function, then any function inside the same file can access that variable - we call this a global variable
- Things get weird if you try to assign a new value inside a function to a variable that was created outside of that function

Shadowing

- When you assign a value to a variable both inside and outside of a function, python treats this as two separate variables with the same name (one local and one global) - this is called shadowing
- If you want to avoid shadowing, you can use the global keyword

Why does scope exist?

- Prevent the programmer from accidentally modifying variables in other parts of the code
- Namespace management - You can use the same variable name in different places without conflict
- Memory management - Local variables are automatically destroyed when the function finishes executing

Classes and Objects

- You have seen data types such as integers, strings, and lists
- Classes allow you to create your own data type
- `class <Classname>:`
 `<contents>`
- To create a class with no contents, use the `pass` keyword
- Objects are specific instances of a class, in the same way that 5 and 6 are specific instances of integers

Attributes

- Objects can have their own variables - we call these attributes
- `my_obj.var_name = "new value"`
- In `basic_class.py` let's try giving the objects of the dog class some attributes
- Attributes share their object's scope even when created in a different scope from the object
- If `y` is an object, and you set a variable `x` to equal `y`, then these are the same object - anytime you change `y`, it will also change `x` and vice versa

Methods

- Not only can objects have variables, objects can also have functions - we call these methods
- We have already seen examples of methods when dealing with the turtle module `my_turtle.shape("turtle"); my_turtle.color("green")`
- The first parameter of any method is the object that is being referenced; by convention we name this first parameter `self`
- Methods are typically defined inside the class body, so that all objects of the same class have the same methods

The `__init__` constructor

- Whenever you create a new object of any class, if a method named `__init__` has been defined, then it is called
- This is typically used to set the default values for all attributes of the object
- If you follow this best practice, then anybody reading your code can understand what attributes any object of your class has
- The way we initially created the `Dog.name` attribute outside of the class definition can cause problems if you try to access an attribute before it has been created
- Defining `__init__` with multiple parameters allows you to make more complicated constructors

The `__str__` method

- Like `__init__`, `__str__` is an automatically invoked special method
- `__str__` only has one parameter, which is `self`, and must always return a string
- `__str__` is invoked whenever an object is passed to `print` as an argument, or when casting an object to a string

Everything is an object

- In python every data type, including the ones we have already seen (integers, strings, and lists), are objects
- For example, integers have the `bit_length()` method
- Even functions are objects - which means you can put a function into a variable, or pass a function as an argument into a function
- The “everything” in “everything is an object” does not include statements.

Elements of a list

- We have already seen a list before (list of strings used for colors)
- Lists can have mixed data types (`my_list = [1, 2, "green"]`)
- Individual list elements can be accessed by `my_list[<num>]`
- Lists are zero-indexed - `my_list[0]` is the first element of `my_list`
- Using a negative number grabs an element from the end of a list - `my_list[-1]` is the last element of `my_list` and `my_list[-2]` is the second to last element of `my_list`, etc...
- `del my_list[<num>]` removes that item from a list and shifts everything after it to accommodate
- If `y` is a list, and you set a variable `x` to equal `y`, then these are the same list - anytime you change `y`, it will also change `x` and vice versa, just like with objects

List methods

- `len(my_list)` - returns the number of elements of `my_list`
- `my_list.append(var)` - appends a new element to the end of `my_list` with value equal to `var`
- `my_list.insert(pos, var)` - inserts a list item at position `pos` of `my_list` with value equal to `var`, and moves every element that was at that position or later 1 over
- `my_list.clear()` - removes all elements from `my_list` and sets the number of elements equal to zero
- `my_list.remove(var)` - removes the first item from `my_list` whose value is `var`
- `my_list.copy()` - returns a new different list that is a copy of `my_list`

List Slicing

- Slicing will allow us to access certain sub-lists from the list
- `my_list[<start>:<end>:<step>]`
 - All three numbers are optional
 - `<start>` - index to begin the slice (inclusive) - defaults to 0
 - `<end>` - index to end the slice (non-inclusive) - defaults to length of list
 - `<step>` - interval between elements of slice - defaults to 1
- `new_list = my_list[<slice>]` creates a new list with elements equal to those of the slice
- `del my_list[<slice>]` removes items of the slice from the list and shifts the remaining elements to accommodate
- `my_list[<slice>] = <iterable>` Replaces slice of the list with the iterable. If step size is not 1, length must match.

For loops

- `for <item> in <iterable>:`
 <statement>
- <iterable> is usually a list, but can also use strings
- For loops do not use a test expression to loop like while loops do, but instead iterate over each item in a list
- Python does not have traditional for loops; for loops in python are similar to foreach loops in other languages
- When used with `range()`, they act similar to traditional for loops
- `range(<num>)` counts up from 0 to <num>, not including num